Branching and boundary tests

Learn to understand, design, build, test and explain AI systems responsibly.

Project: Python readiness portfolio

WORKBOOK · 1 OF 15

Recall and choose your starting point

> Record your starting explanation for Branching and boundary tests before opening notes. Show any later correction separately.

Try the recall before opening notes. Then use a different colour or a labelled second line to correct your explanation.

Boolean conditions

Without opening the old lesson, explain one key idea from Boolean **condition**s. Give an example and a mistake that the idea helps you avoid.

> **My remembered idea, example and correction**

Values variables and types

Without opening the old lesson, explain one key idea from Values variables and types. Give an example and a mistake that the idea helps you avoid.

My remembered idea, example and correction

WORKBOOK · 2 OF 15

Practise with fading help

> Complete the supported Branching and boundary tests trace and record the first step you can explain without help.

Use W2: Trace the tested path for **Branch**ing and **boundary** tests. Record the answer once; the lesson and workbook refer to the same attempt.

Given inputs	Procedure I chose	Middle steps	Result and check

The step I completed without help, and the hint I needed

WORKBOOK · 3 OF 15

Individual reasoning check

> Check the proposed Branching and boundary tests explanation against the supplied facts

> before discussing it.

Use the worked case for **Branch**ing and **boundary** tests. Proposed explanation: “Every **condition** in the chain always runs”. Decide whether this explanation is supported.

Point to the step or supplied fact that decides; explain your repair if needed.

> **My explanation before partner discussion**

WORKBOOK · 4 OF 15

Independent transfer case

> Write one first attempt at the reserved independent case before feedback. Repeated copies of that case refer to the same recorded attempt.

Independent case: Trace branch order

Predict the action for 30, then trace 29 and -1. Explain why validity is checked before the lamp decision.

```
reading = 30
if not 0 <= reading <= 100:
    action = "invalid"
elif reading < 30:
    action = "on"
else:
    action = "off"
print(action)
```

My independent explanation, trace and limits

Attempt this case before feedback. Record any help. Earlier examples below are further practice; a case already practised with feedback cannot establish fresh transfer.

WORKBOOK · 5 OF 15

W1 Explain the chain

> Explain the chain. Apply the supplied Branching and boundary tests facts and keep your reasoning.

Describe the purpose of if, elif and else in the worked duration sorter. Explain why print(label) is outside the indented **branch**es.

My reasoning and evidence

WORKBOOK · 6 OF 15

W2 Trace the tested path

> Trace the tested path. Apply the supplied Branching and boundary tests facts and keep your reasoning.

For -1, 12 and 13, record checked **condition**s, skipped **condition**s and the final label. Run each as a fresh script after predicting.

> **My reasoning and evidence**

WORKBOOK · 7 OF 15

W3 Repair the overwritten result

> Repair the overwritten result. Apply the supplied Branching and boundary tests facts and keep your reasoning.

Trace the two-independent-if example for -1. Identify which assignment overwrites the earlier label and make the smallest repair.

> **My reasoning and evidence**

WORKBOOK · 8 OF 15

W4 Build a new sorter independently

> Attempt before feedback. If this repeats the selected independent case, reuse that first attempt; otherwise keep a separate record. Build a new sorter independently. Apply the supplied Branching and boundary tests facts and keep your reasoning.

Use an integer tiles count. Label a negative value Invalid, zero Empty, 1 through 6 Small and values above 6 Large. Write an ordered if elif else script and predict -1, 0, 1, 6 and 7. Include one print after the chain.

My reasoning and evidence

W5 Choose revealing regression tests

> Choose revealing regression tests. Apply the supplied Branching and boundary tests facts and keep your reasoning.

A repair changes the small-count **condition** from < 6 to <= 6. Name the case that exposes the original error and two different kinds of earlier cases to rerun.

My reasoning and evidence

WORKBOOK · 10 OF 15

Project checkpoint and reflection

> Connect Branching and boundary tests to Python readiness portfolio, identify a limit and choose the next useful test.

Add a labelled **branch**ing and **boundary** tests **evidence** sheet to Python readiness portfolio. Show how you can trace the selected **branch** in an if elif else chain.

Use the supplied fictional case and keep its **input**s, intermediate steps, calculation or **prediction**, and limitation. Connect this record to last week's record; a matching answer without a trace is insufficient.

My project change, supporting evidence and remaining limit

WORKBOOK · 11 OF 15

Feedback, return check and learning record

> Record the first repaired step and a different case to revisit after a delay.

Evidence	My record
Worked alone / hint / model	
First step I repaired and why	
New case checked later; date and result	

Evidence	My record
What I can now explain without notes	
One remaining question	

Revisit this idea with your teacher after a delay. Familiarity is different from explaining and using it on a changed case.

WORKBOOK · 12 OF 15

Pictorial case practice: Branching and boundary tests

> Check invalid values before branching. A toy shelf label uses fewer than five books as "small", otherwise "large", but negative counts are invalid. Explain the labelled step before checking the changed condition.

Check invalid values before **branch**ing. A toy shelf label uses fewer than five books as "small", otherwise "large", but negative counts are invalid.

Python 3: if count<0: label="invalid"; elif count<5: label="small"; else: label="large". Read as indented **branch**es.

AI ACADEMY / PYTHON-BRIDGE/5

Branching and boundary tests

Start with the supplied facts

Follow each arrow. Ask which fact allows the next step.



1. Check validity

For count=-2, the first condition selects invalid.



2. Check small boundary

For count=4, validity passes and count<5 selects small.



3. Handle equality

For count=5, both earlier conditions fail, so large is selected.

CHANGE ONE THING

Remove the first negative-count branch.

Explain the changed result using the matching lesson.

Follow the labelled picture with the matching explanation. Open the vector diagram

Trace -2, 4 and 5 through the **branch**es. Explain why the negative check comes first and what label -2 gets if it is removed.

My picture-based explanation and changed-case prediction

Record the help used

Support used: alone / hint / worked example. This record is practice **evidence**.

Project connection: For Python readiness before the Responsible Plant Classifier, retain a trace of the values, the predicted **output**s and the changed-case explanation.

Use actual Python 3 results to check the **prediction** after reasoning first.

WORKBOOK · 13 OF 15

Project portfolio milestone

> Keep this week's input, deciding step, result and limit together in the Python readiness portfolio record.

Project: Python readiness portfolio

Current milestone: Python readiness portfolio. Keep runnable code, **prediction**s, actual **output**s, one repaired error and an explanation of a function on changed **input**s.

For Python readiness before the Responsible Plant Classifier, retain a trace of the values, the predicted **output**s and the changed-case explanation. Use actual Python 3 results to check the **prediction** after reasoning first.

Evidence to keep	My record
This week's input and deciding step	
Prediction and actual result (or labelled paper trace)	
One change since the previous checkpoint and why	
One error or limit and the next test	

Explain what this week adds to your project. Reuse last week's record rather than starting again.

WORKBOOK · 14 OF 15

Supported practice and learning evidence

> Try the supported questions before comparing with hints and feedback. Record what you repaired and the evidence you retained.

Use this supported practice once, in the browser or on paper. Keep the reserved independent assessment for a separate first attempt before teacher feedback.

Check invalid values before **branch**ing. A toy shelf label uses fewer than five books as "small", otherwise "large", but negative counts are invalid.

Practice 1: explain the deciding evidence

Which labelled step matches this detail? For count=5, both earlier **condition**s fail, so large is selected.

1. Check validity

2. Handle equality

3. Check small **boundary**

My selected conclusion, deciding fact and explanation

Hint · try first

Hint if needed: Read the steps in the pictorial case. Match the supplied detail to the stage that performs or records it.

Feedback · compare after trying

After your attempt, compare with the supported explanation: Handle equality For count=5, both earlier **condition**s fail, so large is selected. An if/elif/else chain selects the first matching **branch**.

Putting invalid handling first prevents a negative count from being mislabelled as a small valid collection.

Practice 2: explain the deciding evidence

Changed case · predict first

Changed-case practice: Remove the first negative-count **branch**. Which conclusion is supported by the supplied facts?

1. This one example guarantees correct results for every future **input**.

2. A count of -2 would incorrectly receive small.

3. We can decide the result without examining the changed **input** or the procedure.

My selected conclusion, deciding fact and explanation

Hint · try first

Hint if needed: Use the changed **condition** and the deciding step in the pictorial trace. The answer must say what follows from those facts.

Feedback · compare after trying

After your attempt, compare with the supported explanation: A count of -2 would incorrectly receive small. Negative two is less than five, but it is not a permitted measured count.

Repair a missing building block

Review Boolean **condition**s (python-bridge/4). Goal: Translate stated requirements into comparisons. Short practice: Give both comparison results for ages 11, 15 and 16.

Which comparison rejects each outside value, and why does and matter?. Return to this week and explain what you changed.

Review Values variables and types (python-bridge/2). Goal: Distinguish integer, floating-point, text and Boolean values. Short practice: Predict both print **output**s and explain why they differ.

Does count+tag work, and what happens to each valid **output** after count becomes ten?. Return to this week and explain what you changed.

Predict, run and debug

The browser lab lets you edit the supported Python examples already included in this lesson. Predict first, run, inspect the actual **output** or error, change one **input**, then run again.

Keep code, **prediction**s, actual results and a reasoned revision together. On paper, label your result as a trace rather than an execution.

Keep your completed work

Use Download completed work in the hosted lesson to export your answers, reflections, practice feedback and code-run **evidence**. Open the HTML copy to read or print it.

Drafts stay in this browser when storage is available; clear drafts on a shared device.

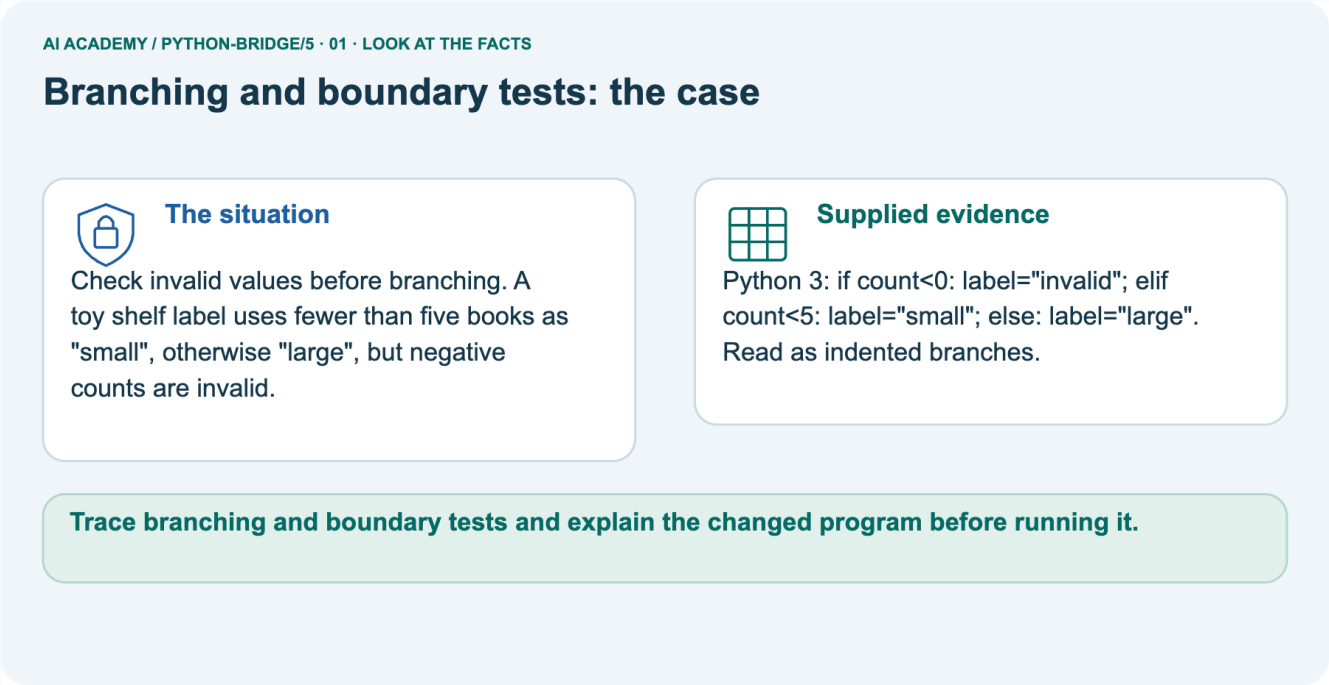
> **What I repaired, the evidence I kept and the next test**
>
> | |
> |---|
> | |
> | |
> | |
> | |

WORKBOOK · 15 OF 15

Visual concept journey

> Explain each Branching and boundary tests picture from its labels, then record a relevant limit.

01. Branching and boundary tests: the case



Follow the labelled picture with the matching explanation. Open the vector diagram

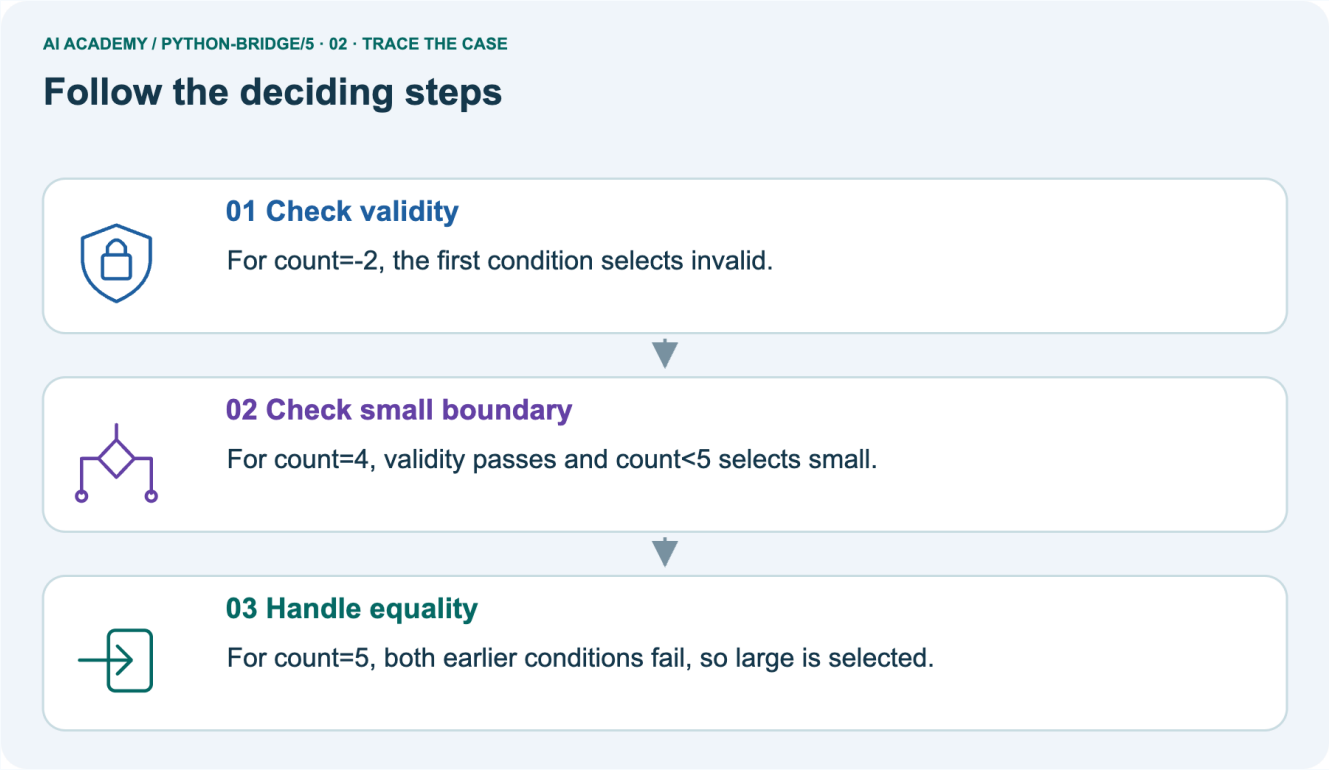
Check invalid values before **branch**ing. A toy shelf label uses fewer than five books as "small", otherwise "large", but negative counts are invalid.

Supplied facts: Python 3: if count<0: label="invalid"; elif count<5: label="small"; else: label="large". Read as indented **branch**es.

> Point to the given fact that matters most. Explain why it matters.

My picture explanation, deciding evidence and one limit

02. Follow the deciding steps



Follow the labelled picture with the matching explanation. Open the vector diagram

Check validity: For count=-2, the first **condition** selects invalid. → Check small **boundary**: For count=4, validity passes and count<5 selects small. → Handle equality: For count=5, both earlier **condition**s fail, so large is selected.

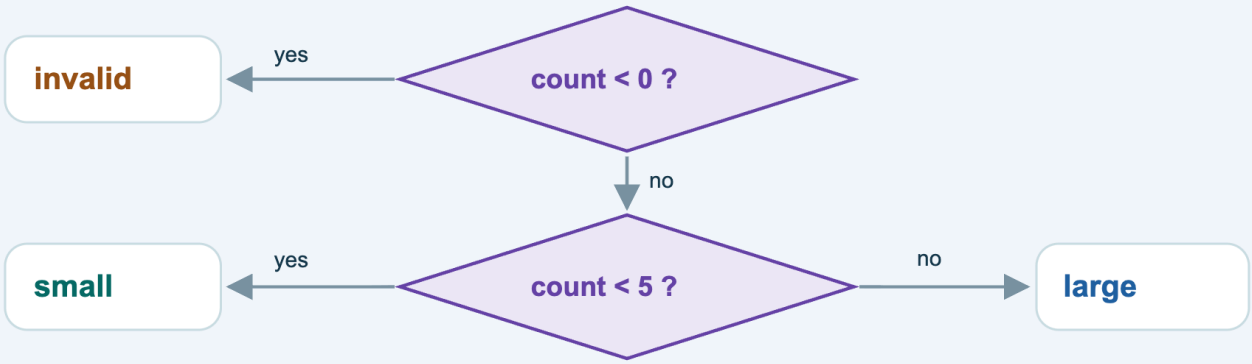
> Point to each numbered stage and explain which recorded **input** it uses.

My picture explanation, deciding evidence and one limit

03. Check invalid values before the size rule

AI ACADEMY / PYTHON-BRIDGE/5 · 03 · FOLLOW THE FIRST MATCHING BRANCH

Check invalid values before the size rule



Trace separate inputs through the same procedure:

−2 → invalid | 4 → small | 5 → large

A chosen branch ends the chain. Equality with 5 reaches the “large” branch.

Follow the labelled picture with the matching explanation. Open the vector diagram

First ask count less than zero: yes selects invalid. Otherwise ask count less than five: yes selects small; no selects large. Separate **input** traces are minus two invalid, four small, five large.

Read the labels and values. Explain the relationship the model shows.

My picture explanation, deciding evidence and one limit

04. Change one thing. Trace what follows.

AI ACADEMY / PYTHON-BRIDGE/5 · 04 · COMPARE THE CASES

Change one thing. Trace what follows.

Original case

For count=5, both earlier conditions fail, so large is selected.

Changed condition

Remove the first negative-count branch.

Worked changed-case result

A count of -2 would incorrectly receive small.
Why: Negative two is less than five, but it is not a permitted measured count.

Follow the labelled picture with the matching explanation. Open the vector diagram

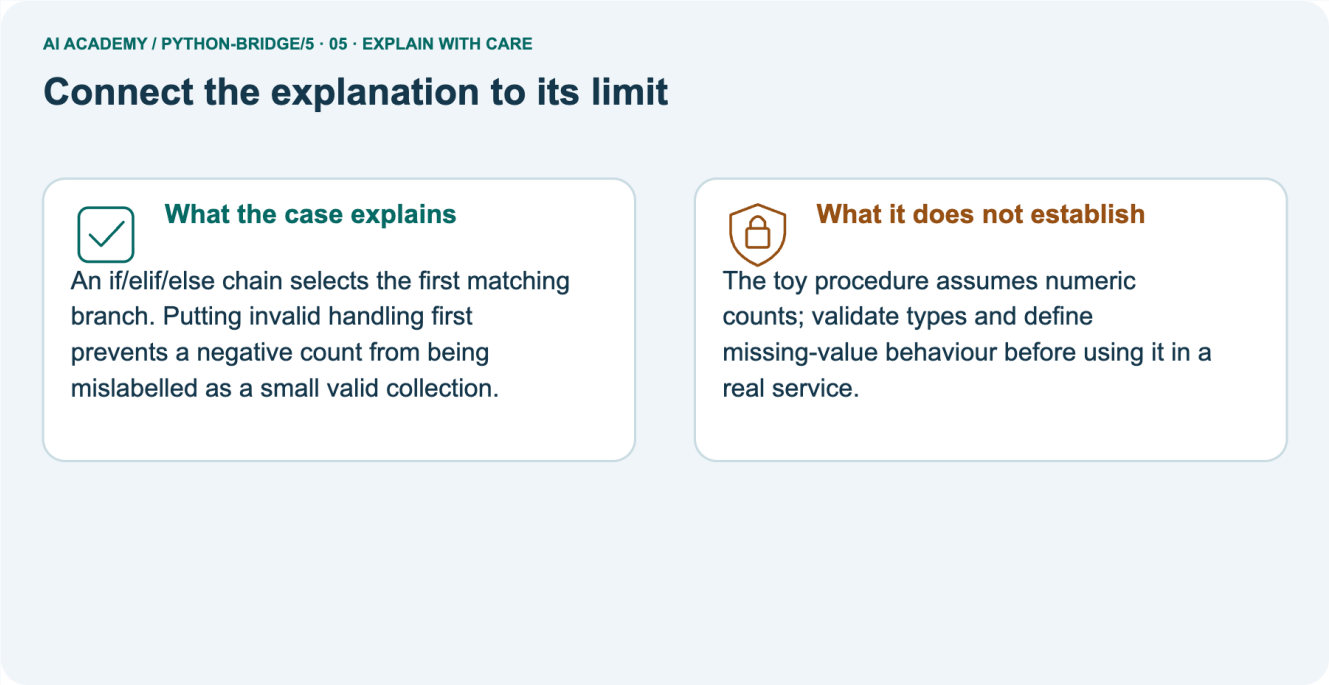
Original case: For count=5, both earlier **condition**s fail, so large is selected. Changed **condition**: Remove the first negative-count **branch**. Expected result: A count of -2 would incorrectly receive small.

Reason: Negative two is less than five, but it is not a permitted measured count.

> Predict before checking the worked changed-case result. Name what changed and what the supplied facts allow you to conclude.

My picture explanation, deciding evidence and one limit

05. Connect the explanation to its limit



Follow the labelled picture with the matching explanation. Open the vector diagram

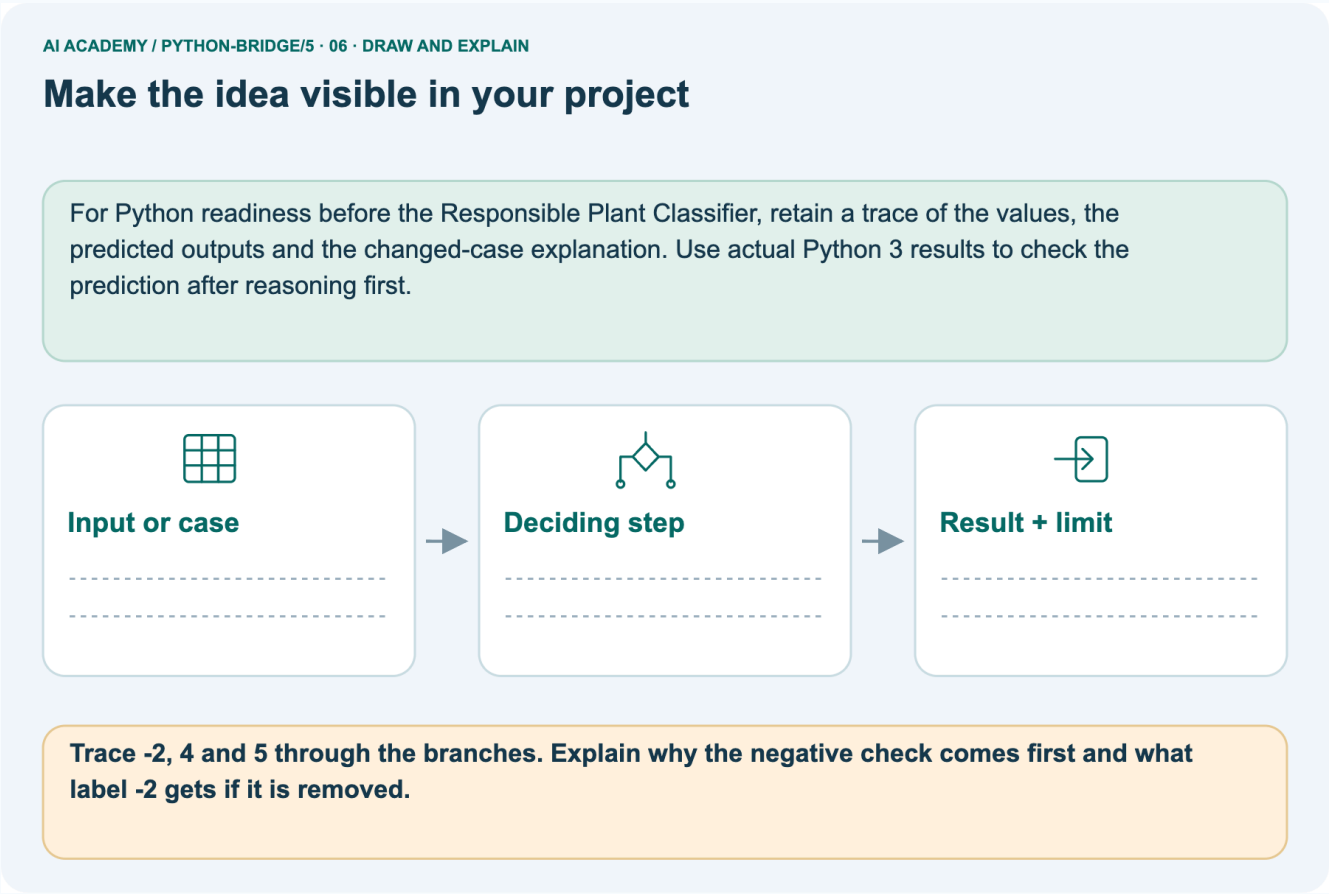
An if/elif/else chain selects the first matching **branch**. Putting invalid handling first prevents a negative count from being mislabelled as a small valid collection.

Limit: The toy procedure assumes numeric counts; validate types and define missing-value behaviour before using it in a real service.

> Explain the case, then point to a limit on the claim.

My picture explanation, deciding evidence and one limit

06. Make the idea visible in your project



Follow the labelled picture with the matching explanation. Open the vector diagram

For Python readiness before the Responsible Plant Classifier, retain a trace of the values, the predicted **output**s and the changed-case explanation. Use actual Python 3 results to check the **prediction** after reasoning first.

Draw your **input**, deciding step, result and limit. Supported question: Trace -2, 4 and 5 through the **branch**es. Explain why the negative check comes first and what label -2 gets if it is removed.

> Draw your project trace and justify the deciding step using **evidence**.

My picture explanation, deciding evidence and one limit

Save or print your responses for your portfolio. Progress and drafts on the hosted website stay in the current browser. Use only fictional or teacher-approved data.